前書き

本書の内容は、特定の実装や過去の判断を批判するものではありません。

mocochro のカスタマイズは、その時々の要件を確実に満たすために、限られた時間と情報の中で最善の判断として積み重ねられてきたものです。

私自身、Chromium・Ninja・mocochro カスタマイズを深く理解していない状態から開発を始め、複数のバージョンアップ（v93→v118、v128 失敗）や macOS / Linux / Android への移植を通じて、現在の方式が **Chromium の進化に伴って追従しにくくなっている** ことを実感しました。

以下は、今後の保守性・開発効率を高めるための改善ポイントを、技術的な観点から整理したものです。

改善ポイント

1. 侵入型（Intrusive）カスタマイズによる追従の難しさ

mocochro の多くの機能は、Chromium の内部関数に直接コードを追加する形で実装されています。

これは分かりやすい反面、Chromium のバージョンアップ時に影響を受けやすい構造です。

● バージョンアップで実際に起きる現象（技術詳細）

- Chromium 本体の関数・ファイルが **移動・改名・削除** される
- 同じ名前の関数でも **内部処理が変更** されている
- 処理経路そのものが変わり、過去のパッチが **呼ばれなくなる**
- v118 で存在したロジックが、v141 では **完全に異なる仕組みに置き換わっている**

そのため、**過去にパッチを入れていた同じ場所が、新バージョンで適切な場所とは限らない**という状況が発生します。

v141 のコードフローを理解し直し、その中で正しい位置に機能を再配置する必要があり、移植ではなく**実質的な再設計に近い作業**になります。

---

2. 基盤クラスへの拡張による依存範囲の巨大化

過去の開発では、以下のような基盤クラスへフィールド追加やメソッド拡張が行われてきました：

- NavigationRequest
- URLLoader

- RenderFrameHost
- WebContents
- Network 構造体
- Prefs
- IPC 関連構造体

これにより要件を実現してきましたが、依存が増えることで次のような現象が発生します。

● 依存チェーンが生む技術的負荷

- 1 箇所の変更で **20 ~ 50 ファイル以上が同時にビルドエラー**
- 「まず全箇所を移植してビルドを通す」必要があり、問題の切り分けが遅れる
- ビルドが通るまで **動作確認ができない**
- v128 では、

  → 全部移植後に **新しい Chromium フローと構造的に一致しないため動かない** ことが後から判明

つまり、**密結合（tight coupling）が年々進み、バージョンアップの負荷が加速度的に増している**という状況です。

総合的な判断

現状の構造のままでは、v141 の移行には大きな工数とリスクが伴います。

さらに、次回以降のバージョンアップでも同じ問題が繰り返される可能性が高いと考えられます。

これは実装が誤っていたわけではなく、**巨大で変化の早い Chromium と、長年の機能追加による構造的負荷の積み重ねによるもの** です。

リファクタリングの目的

目的は、mocochro のカスタマイズをできるだけ**Chromium の内部ロジックに依存させず、拡張ポイント（Hook/Observer/Delegate）から扱える構造に整理すること**です。

● 重視する改善点

- 基盤クラスへのメソッド直接追加の削減
- 内部処理への直接挿入を避ける
- デフォルト挙動の書き換えの最小化
- カスタム処理の集中管理

- Hook / Delegate / Observer / Factory を最大限活用する構造への移行

これにより、

- 影響範囲が小さくなる
- バージョンアップでコードが壊れにくくなる
- カスタマイズ箇所の特定が容易になる
- 開発チーム全体で同じ設計方針を共有できる

といった効果が期待できます。

優先的に改善すべき領域

特に以下の領域は、依存や変更範囲が広く、改善効果が大きいと考えています。

1. Portal WebUI

- コードが分散しており、フロー理解が困難
- v118 と v141 で WebUI データソースや処理フローが大きく異なる

2. Prefs

- Value API の変更、raw_ref の追加など、v141 で仕様が大きく変化
- 過去拡張との衝突が発生しやすい

3. Network 層

- URLLoaderFactory や SimpleURLLoader のフロー変更
- Gateway ロジックが絡むため影響範囲が広い

4. Custom UI (ダイアログ / ウォーターマーク)

- Navigation や WebContents のフロー変更と密接に結びついている

推奨する進め方

理想的なアプローチ

v118 上でリファクタし、→ その後 v141 に移行する方法。理解度が高い v118 で整理できるため、最も安全で確実です。

現実的なアプローチ

現在採用している**リファクタリングと v141 移行の並行作業**も有効です。

短期的負荷はありますが、長期的には**移行成功率向上・保守性向上・今後のアップグレード容易化**の効果が期待できます。

コードレビューの強化

レビューは批判ではなく、

知識共有とリスクの早期発見のための仕組み として行う。

- コミットごとに複数人でレビュー
- 設計方針・保守性・影響範囲を確認
- カスタマイズの追加位置が適切かチェック

総括

現状の構造のままでは、バージョンアップごとに大きな工数が発生し、開発負荷が増大し続ける可能性があります。

しかし構造を整理することで、

- バージョンアップが大幅に簡単になる
- 不具合の切り分けが容易になる
- カスタマイズが壊れにくくなる
- 新規機能追加がより安全になる
- チーム全体で共通理解を持てる

という大きな効果が得られます。

これは過去の実装を否定するものではなく、**未来の開発をより良くするための前向きな改善**として位置づけたいと思います。